

SIMATS Engineering • CSE • CSA0212 — C PROGRAMMING • 04-08-2026 • Category: CO2-AT2-SCENARIO BASED QUESTIONS (MEDIUM) • Student Submissions Report

Student 22 **PILLETI HEMANTH REDDY** [192312173RB](#) 4 / 20 submitted

Q1. Arrays

CODE

```
#include <stdio.h>

int main() {
    int month = 0;
    int production[12];

    // Inputs for 12 months
    printf("Enter production for 12 months (Jan to Dec). \n");
    printf("Enter a negative value to represent invalid production.\n");
    printf("Enter -999 to simulate equipment failure (plant shutdown).\n\n");

    for (month = 0; month < 12; month++) {
        printf("Month %d production: ", month + 1);
        scanf("%d", &production[month]);

        if (production[month] == -999) {
            // Plant shutdown due to equipment failure
            break;
        }
    }

    int monthsCounted = month; // number of months processed before break
    if (monthsCounted == 0) {
        printf("\nPlant shut down immediately. No production data to analyze.\n");
        return 0;
    }

    // (a) Arithmetic operators: annual total, average monthly, efficiency
    int annualProduction = 0;
    for (int i = 0; i < monthsCounted; i++) {
        // (b) continue: skip invalid production values (negative except -999)
        if (production[i] < 0) {
            continue;
        }
        annualProduction += production[i];
    }

    // Target per month = 5000, so annual target depends on counted months
    int targetPerMonth = 5000;
    int annualTarget = monthsCounted * targetPerMonth;

    // Average monthly production (only from valid counted months)
    // We'll compute average using another pass to count valid months
    int validMonths = 0;
    int sumValid = 0;
    for (int i = 0; i < monthsCounted; i++) {
        if (production[i] < 0) {
            continue;
        }
        sumValid += production[i];
        validMonths++;
    }

    double averageMonthly = 0.0;
    if (validMonths > 0) {
        averageMonthly = (double)sumValid / validMonths;
    }

    // Production efficiency = (annual actual / annual target) * 100
    double efficiency = 0.0;
    if (annualTarget > 0) {
```

```
        efficiency = ((double)annualProduction / annualTarget) * 100.0;
    }

    // Decision-making for incentive and inspection
    // Incentive: production exceeds target → 10% performance incentive
    // We'll compare annualProduction with annualTarget.
    if (annualProduction > annualTarget) {
        double incentive = 0.10 * annualProduction;
        printf("\n--- Production Summary (Jan to Month %d) ---\n", monthsCounted);
        printf("Annual Production (actual): %d\n", annualProduction);
        printf("Annual Target: %d\n", annualTarget);
        printf("Average Monthly Production: %.2f\n", averageMonthly);
        printf("Production Efficiency: %.2f%%\n", efficiency);
        printf("Status: Incentive eligible (10%% performance incentive).\n");
        printf("Incentive Amount: %.2f\n", incentive);
    } else {
        printf("\n--- Production Summary (Jan to Month %d) ---\n", monthsCounted);
        printf("Annual Production (actual): %d\n", annualProduction);
        printf("Annual Target: %d\n", annualTarget);
        printf("Average Monthly Production: %.2f\n", averageMonthly);
        printf("Production Efficiency: %.2f%%\n", efficiency);

        // Inspection condition: below 80% of target
        if (annualProduction < (0.80 * annualTarget)) {
            printf("Status: Production below 80%% of target → Inspection marked.\n");
        } else {
            printf("Status: No incentive and no inspection.\n");
        }
    }
}

// (b) Arrays processing message for each month (skip invalid using continue)
printf("\n--- Month-wise Valid Production Details ---\n");
for (int i = 0; i < monthsCounted; i++) {
    if (production[i] < 0) {
        continue; // skip invalid months
    }
    printf("Month %d: %d batteries produced\n", i + 1, production[i]);
}

// If break happened (equipment failure)
if (month < 12) {
    printf("\nNote: Plant shutdown occurred due to equipment failure at Month %d.\n", month + 1);
} else {
    printf("\nNote: Data entered for all 12 months.\n");
}

return 0;
}
```

INPUT

12

OUTPUT

(no output)

Q2. Arrays

CODE

```
#include <stdio.h>

int main() {
    int quality[20], delay[20], cancelled[20];
    int i;

    int sumQuality = 0;
    int activeCount = 0;

    int deliveredCount = 0;
    /* Interpreting delivery as meeting the best delivery condition:
       Delay = 2 days (you can change to ≤2 if your lab expects that). */

    for (i = 0; i < 20; i++) {
        int stopSignal;

        printf("Enter stop signal (0=continue, 1=stop) for project %d: ", i + 1);
        scanf("%d", &stopSignal);

        if (stopSignal == 1) {
            break; /* break if management stops evaluation */
        }

        printf("Enter Quality (0-100) Delay(days) Cancelled(0/1) for project %d: ", i + 1);
        scanf("%d %d %d", &quality[i], &delay[i], &cancelled[i]);

        if (cancelled[i] == 1) {
            continue; /* ignore cancelled projects */
        }

        activeCount++;

        /* (a) Classification using decision-making statements */
        printf("Project %d Result: ", i + 1);

        if (quality[i] ≥ 90 && delay[i] == 2) {
            printf("Excellent\n");
        } else if (quality[i] ≥ 75 && quality[i] ≤ 89 && delay[i] == 5) {
            printf("Good\n");
        } else if (quality[i] ≥ 60 && quality[i] ≤ 74) {
            printf("Average\n");
        } else if (quality[i] < 60) {
            printf("Needs Improvement\n");
        } else {
            printf("Not Classified\n");
        }

        /* (a) Use arithmetic operators to compute totals */
        sumQuality = sumQuality + quality[i];

        /* Delivery percentage condition */
        if (delay[i] == 2) {
            deliveredCount++;
        }
    }

    printf("\n--- Summary ---\n");

    if (activeCount > 0) {
        float avgQuality = (float)sumQuality / activeCount;
        float deliveryPercent = ((float)deliveredCount * 100.0f) / activeCount;
```

```
        printf("Active Projects: %d\n", activeCount);
        printf("Average Quality Score: %.2f / 100\n", avgQuality);
        printf("Delivery Percentage: %.2f%%\n", deliveryPercent);
    } else {
        printf("No active projects to evaluate.\n");
    }

    return 0;
}
```

OUTPUT (no output)

Q3. Arrays

— Not Submitted —

Q4. Arrays

— Not Submitted —

Q5. Arrays

— Not Submitted —

Q6. Arrays

CODE

```
#include <stdio.h>

int main() {
    int chipsPerDay = 10000;
    double defectPercentage;
    int defectChips;
    int acceptedChips;
    char grade;

    // Part (a)
    int defects;
    printf("Enter total number of defective chips in a day (out of %d): ", chipsPerDay);
    scanf("%d", &defects);

    defectPercentage = (defects * 100.0) / chipsPerDay;
    acceptedChips = chipsPerDay - defects;

    if (defectPercentage < 1.0) {
        grade = 'A';
    } else if (defectPercentage ≥ 1.0 && defectPercentage ≤ 3.0) {
        grade = 'B';
    } else if (defectPercentage > 3.0 && defectPercentage ≤ 5.0) {
        grade = 'C';
    } else {
        grade = 'R'; // Reject Batch
    }

    printf("\n--- Part (a) ---\n");
    printf("Defect percentage = %.2f%%\n", defectPercentage);

    if (grade == 'R') {
        printf("Batch REJECTED. Accepted chips = 0\n");
    } else {
        printf("Grade = %c\n", grade);
        printf("Accepted chips = %d\n", acceptedChips);
    }

    // Part (b)
    printf("\n--- Part (b) Arrays + continue + break ---\n");
    printf("Enter production/maintenance info for 30 days.\n");
    printf("For each day, enter:\n");
    printf("Type 1 = Maintenance Day (skip), Type 2 = Normal Day, Type 3 = Machine Breakdown (stop)\n");

    int day;
    int defectsArr[30];
    int acceptedArr[30];
    int processedDays = 0;

    for (day = 0; day < 30; day++) {
        int type;
        printf("\nDay %d - Enter type (1/2/3): ", day + 1);
        scanf("%d", &type);

        if (type == 1) {
            // maintenance day → use continue
            continue;
        }

        if (type == 3) {
            // breakdown → stop using break
            printf("Machine breakdown occurred on Day %d. Stopping further processing.\n", day + 1);
        }
    }
}
```

```
        break;
    }

    // Normal Day (type == 2)
    int defectsToday;
    printf("Enter defective chips for Day %d (out of %d): ", day + 1, chipsPerDay);
    scanf("%d", &defectsToday);

    defectsArr[processedDays] = defectsToday;

    defectPercentage = (defectsToday * 100.0) / chipsPerDay;

    if (defectPercentage < 1.0 || (defectPercentage ≥ 1.0 && defectPercentage ≤ 3.0) || (defectPercentage > 3.0 && defectPercentage ≤ 5.0)) {
        acceptedArr[processedDays] = chipsPerDay - defectsToday;
    } else {
        acceptedArr[processedDays] = 0; // Reject batch
    }

    processedDays++;
}

printf("\nProcessed %d normal days.\n", processedDays);
printf("\nAccepted chips for processed days:\n");
for (int i = 0; i < processedDays; i++) {
    printf("Day %d → Accepted chips = %d\n", i + 1, acceptedArr[i]);
}

return 0;
}
```

OUTPUT (no output)

Q7. Arrays

CODE

```
#include <stdio.h>

int main() {
    int n;
    printf("Enter number of customers: ");
    scanf("%d", &n);

    int basic = 100;
    int standard = 500;
    int enterprise = 2000;

    float extraRate = 4.0;          // Rs. 4 per GB
    float warningThreshold = 0.95; // 95%

    int serverCapacity = 2000;      // total server storage (GB)
    int totalUsed = 0;

    int active[100], plan[100], used[100]; // fixed arrays for safety

    printf("Input format: active(1/0) plan(1=Basic,2=Standard,3=Enterprise) usedStorage(GB)\n");
    for (int i = 0; i < n; i++) {
        printf("Customer %d: ", i + 1);
        scanf("%d %d %d", &active[i], &plan[i], &used[i]);
    }

    for (int i = 0; i < n; i++) {
        if (active[i] == 0) {
            printf("Customer %d inactive → continue\n", i + 1);
            continue; // requirement
        }

        int planSize;
        if (plan[i] == 1) planSize = basic;
        else if (plan[i] == 2) planSize = standard;
        else planSize = enterprise;

        int usedStorage = used[i];

        // server full condition (break when cannot accommodate this customer)
        if (totalUsed + usedStorage > serverCapacity) {
            printf("Server storage full → break at Customer %d\n", i + 1);
            break; // requirement
        }

        int remainingInPlan = planSize - usedStorage;
        int extraGB = 0;

        if (remainingInPlan ≥ 0) {
            extraGB = 0;
        } else {
            extraGB = -remainingInPlan;
            remainingInPlan = 0;
        }

        float extraCharge = extraGB * extraRate;

        // update server usage after passing full check
        totalUsed = totalUsed + usedStorage;

        float usagePercent = (float)totalUsed / (float)serverCapacity;
    }
}
```

```
printf("\nCustomer %d Details:\n", i + 1);
printf("Plan size (GB) = %d\n", planSize);
printf("Used storage (GB) = %d\n", usedStorage);
printf("Remaining within plan (GB) = %d\n", remainingInPlan);
printf("Extra storage (GB) = %d\n", extraGB);
printf("Extra charges (Rs.) = %.2f\n", extraCharge);

if (usagePercent > warningThreshold) {
    printf("WARNING: Server usage exceeded 95% (Current: %.2f%%)\n", usagePercent * 100.0f);
} else {
    printf("Server usage: %.2f%%\n", usagePercent * 100.0f);
}
}

printf("\nFinal Server Used Storage = %d GB out of %d GB\n", totalUsed, serverCapacity);
return 0;
}
```

OUTPUT (no output)

Q8. Arrays

— Not Submitted —

Q9. Arrays

— Not Submitted —

Q10. Arrays

— Not Submitted —

Q11. Arrays

— Not Submitted —

Q12. Arrays

— Not Submitted —

Q13. Arrays

— Not Submitted —

Q14. Arrays

— Not Submitted —

Q15. Arrays

— Not Submitted —

Q16. Arrays

— Not Submitted —

Q17. Arrays

— Not Submitted —

Q18. Arrays

— Not Submitted —

Q19. Arrays

— Not Submitted —

Q20. Arrays

— Not Submitted —